inBox

E-COMMERCE PLATFORM

Project evaluation and technical report

A source-based guide for managers, project judges, developers and peers.

REPORT SNAPSHOT	DETAIL
Prepared for	Mayank Godhani and project reviewers
Review date	28 October 2026 (India Standard Time)
Project	inbox-ecommerce-store | package version 0.0.0
Source revision	91ac469 | working tree clean before report work
Scope	14 page components, 13 backend route modules, 71 HTTP method/path registrations, 17 Prisma models
Evidence	Current source, fresh checks, saved browser results, historical performance narrative and CodeAnt JSON

Assessment: A substantial commerce implementation with customer, merchant and platform administration workflows. Build and type checks pass. Production approval remains conditional on payment integrity, seller authorization, schema migrations and independent integration verification.

No passwords, connection secrets, session tokens or customer records are reproduced in this report.

01 Project introduction

inBox is a full-stack e-commerce application that brings shopping, order management, merchant operations and seller governance into one browser-based platform. Customers can browse a catalog, search and filter products, choose variants, save favorites, manage a cart and address book, obtain a server-calculated quote and place orders. Merchants receive a dedicated operations panel, while super administrators review seller applications and govern platform access.

The project addresses a common commerce problem: product discovery, pricing, stock, customer accounts and fulfillment must remain consistent across multiple screens and user roles. Its architecture separates a React presentation layer from an Express API and PostgreSQL persistence. Prisma describes the relational model and supplies typed database access. The backend recalculates monetary values from stored product prices and reserves inventory through conditional updates inside transactions.

Business purpose and intended users

For shoppers, the platform provides a journey from product discovery to order tracking. For merchants, it centralizes inventory, imagery, fulfillment queues, customer information and revenue indicators. For platform operators, it provides seller application review, approval, rejection, suspension and administrator governance. Managers and judges can evaluate both the implemented workflows and the engineering evidence behind them.

Implemented scope

The interface includes a storefront, catalog, product details, cart, wishlist, checkout, order success and tracking, order history, profile, login, seller onboarding, merchant dashboard and super-admin dashboard. English, German and Dutch translation dictionaries are present. Stripe hosted checkout and Cash on Delivery use separate payment paths. Cloudinary supports product-image uploads when configured.

Engineering approach

The application uses browser-history navigation, React contexts for shared state, a fetch-based API client, cookie-backed sessions, role checks, REST-style modules, database constraints, and unit, mocked-service, component, browser and load-testing tools. The source includes Docker configurations, seed and backup utilities, and a GitHub Actions verification workflow.

Evaluation position

The implementation is suitable for a detailed project demonstration and engineering review. This report does not certify a production release: source inspection found payment-trust and seller-scope gaps, schema-to-migration drift and incomplete recovery evidence. Fresh test outcomes and older results are separated so reviewers can judge the actual state of the project.

Evidence: src/App.tsx; server/app.ts; prisma/schema.prisma; package.json; server/modules/*

02 Reading guide and evidence

SECTION	WHAT IT ANSWERS
01-02 | Introduction and evidence Pages 2-3	Project purpose and basis for claims.
03-04 | Technology and architecture Pages 4-5	Frameworks, tools, boundaries and integrations.
05-07 | Panels, routes, workflows Pages 6-8	User roles, screen addresses and commerce behavior.
08-12 | Database and ER diagrams Pages 9-17	Models, fields, keys, relationships and migration caveats.
13-14 | APIs and examples Pages 18-23	Every registered HTTP operation, access rule and key contracts.
15-17 | Tests and analysis Pages 24-27	Fresh checks, saved Playwright results, k6 claims and CodeAnt scan.
18-20 | Operations Pages 28-32	Setup, deployment, configuration, recovery and commands.
21-22 | Evaluation and sources Pages 33-36	Release priorities, demonstration guide and source index.

Evidence labels

LABEL	MEANING
Verified now	Command executed in this review; output retained under tmp/pdfs/.
Source-confirmed	Behavior inspected in current source; not necessarily a live integration result.
Saved result	Existing machine-readable output inspected; no fresh execution implied.
Historically reported	Figure in earlier Markdown documentation; raw performance export unavailable.
Not verified	Not exercised here, or evidence insufficient.

Current source and schema take precedence over older narrative reports. Locked package versions are taken from package-lock.json. A declared dependency is not proof of an active feature. No database migration, seed, charge, load campaign or customer-data mutation was performed for this report. PDF bookmarks provide section navigation.

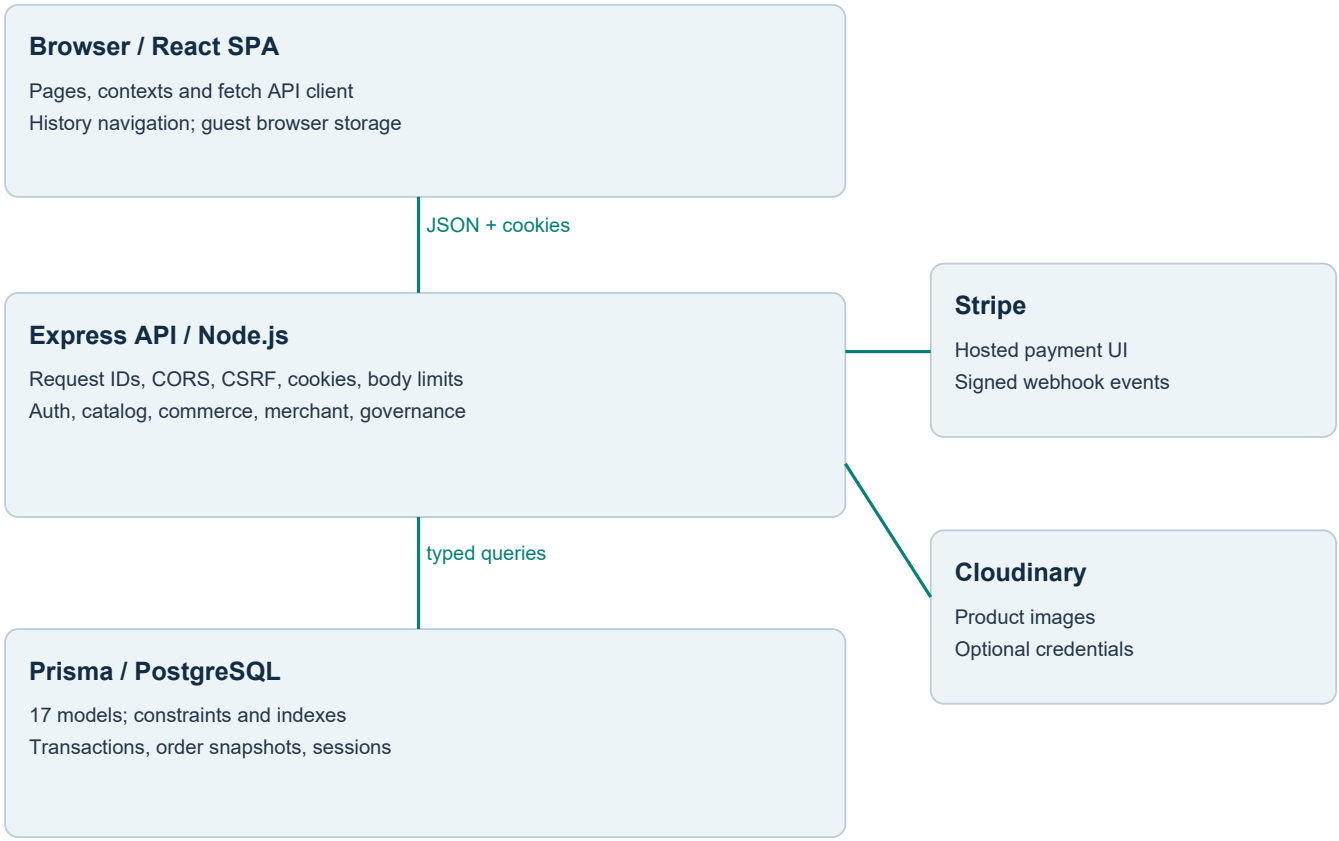
03 Technologies and tools

LAYER	LOCKED VERSION	USE
UI runtime	19.3.1	Component storefront and panels
DOM renderer	19.3.1	Browser rendering
Frontend build	8.3.4	Dev server and production assets
Language	7.0.2	Frontend/backend checks
Styling	4.3.3	Utility CSS
Backend	4.22.3	Modular HTTP API
Database client	6.19.3	PostgreSQL queries and transactions
Schema tools	6.19.3	Migrations, generation and Studio
Passwords	3.0.3	Password hashing and verification
Payments	23.0.0	Hosted sessions and webhooks
Media	2.11.0	Image upload and delivery
Multipart	2.5.0	Image upload parsing
Motion / icons	12.43.0	UI animation; Lucide provides icons
Fast tests	5.0.5	Unit, service, component, regression
Browser tests	1.64.1	Chromium scenarios
DOM assertions	16.3.3	React component testing
HTTP tests	7.3.1	App-level request tests
Server build	0.28.2	Node ESM bundle
Dev runner	4.23.15	TypeScript execution/watch

Node requirement: >=22.20.0 in package.json. Review host: Node 23.5.0, npm 11.0.0, Windows. Docker uses node:22-alpine; CI uses Node 24. Compose declares postgres:18-alpine. k6 scripts and a Windows binary are present. Git/GitHub, Docker Compose and Prisma Studio support operations. MSW, jsdom and V8 coverage packages support testing.

Evidence: package.json; package-lock.json; Dockerfile; compose.yaml; GitHub Actions workflow

04 System architecture



Request and data flow

The browser calls relative /api URLs with credentials included. In development, Vite on port 3000 proxies traffic to Express on port 4000. Express creates/propagates x-request-id, logs duration/status, enforces request policy, parses cookies and bounded bodies, and dispatches routers. Prisma accesses PostgreSQL. Most successful responses use a data envelope; errors use an error object.

Deployment shape

Express serves an existing dist/ directory with an SPA fallback. Docker builds UI and backend in a builder stage and runs Node as an unprivileged user. This is a modular monolith; no separate microservices, queue, Redis cache or background worker was found.

Documentation corrections: navigation uses pushState/popstate, not React Router; the API client uses fetch, not Axios. Helmet is neither listed nor mounted. The @google/genai dependency does not establish an implemented AI workflow.

Evidence: src/App.tsx; src/services/apiClient.ts; vite.config.ts; server/app.ts; Dockerfile

05 Panels and responsibilities

PANEL	IMPLEMENTED CAPABILITIES	BOUNDARY
Customer	Promotions, catalog, product variants/reviews, cart, saved items, wishlist, checkout, orders, tracking and profile.	Public browsing; account needed for persisted personal data and orders.
Merchant / admin	Home overview, Orders, Products, Customers, Financial Analytics; create/edit/archive products, upload images, fulfill orders, collect COD.	ADMIN and SUPER_ADMIN pass the admin guard. Product lists use seller metadata.
Super admin	Seller Applications, Admins & Sellers, System Architecture; approve/reject/suspend/reinstate applications; govern roles and view metrics.	SUPER_ADMIN required after onboarding/private-status routes.
Seller onboarding	Business name, legal name, email, phone, address, category, optional tax ID/description; account linking/creation.	Existing account ownership checked by current session or password.

Roles and seller lifecycle

Database roles are CUSTOMER, ADMIN and SUPER_ADMIN; UI values are customer, admin and superadmin. SellerStatus is separate: PENDING, APPROVED, REJECTED or SUSPENDED. Approval promotes the linked user to ADMIN. Suspension archives products matched by specifications.sellerId or specifications.sellerEmail.

What the metrics represent

Merchant paid revenue sums that merchant's item totals from paid orders; it is not net settlement revenue and does not allocate order-level tax, shipping or discounts. Low stock means stock <=10 for unarchived products. The analytics tab contains fixed monthly revenue bars and a fixed +24% YoY label. Present these as sample UI data, not measured business results.

Seller scope is incomplete: order status/payment mutations check admin role without a merchant-ownership condition; matching order lists include every order item. Suspension is not checked by requireAuth/requireAdmin. These are release-priority findings, not verified protections.

Evidence: src/pages/AdminDashboard.tsx; SuperAdminDashboard.tsx; SellerOnboard.tsx; server/modules/admin; server/modules/superadmin; server/middleware/auth.ts

06 Frontend routes and screens

SCREEN	SUPPORTED ADDRESS	PURPOSE
Home	/	Promotions/catalog entry
Products	/products; /?tab=products	Category/search/filter
ProductDetails	/?tab=product-details&id=PRODUCT_ID	Details/reviews/variants
Cart	/cart; /?tab=cart	Cart and saved items
Wishlist	/?tab=wishlist	Favorite products
Checkout	/checkout; /?tab=checkout	Address, quote, payment
OrderSuccess	/?tab=order-success&orderId=ORDER_NUMBER	Confirmation/Stripe return
Orders	/orders; /?tab=orders	Order history
OrderTracking	/order-tracking?orderId=ORDER_NUMBER	Status and tracking
Profile	/profile; /?tab=profile	Personal data/addresses
Login	/login?redirect=TARGET; /?tab=login	Login and registration
SellerOnboard	/sell	Seller application
AdminDashboard	/admin	Merchant operations
SuperAdminDashboard	/super-admin	Platform governance

Most navigation emits query-tab URLs. Dedicated paths are parsed for admin, super-admin, sell, login, checkout, cart, orders, order-tracking, profile and products. Product lists accept category/search and order screens use orderId. Unknown paths normally fall back to the home tab rather than a dedicated client 404 screen.

Shared components and state

Navbar, Footer, BrandLogo, LanguageSelector, QuickViewModal, RatingStars, ProductCard, FilterSidebar and AuthModal form the reusable interface. Providers compose Language, Toast, Auth, Cart and Wishlist state. Guest state and language preference use browser storage; authenticated commerce data is synchronized through service functions.

Evidence: src/App.tsx; src/components; src/context; src/services

07 Commerce workflows and rules

Browse, quote and checkout

Catalog search uses case-insensitive contains over product name, description and brand, with category equality filtering. Paging defaults to 50 and caps at 100. Sort values: featured, price_asc, price_desc, rating, newest. This is database filtering, not a dedicated full-text search engine.

The quote service accepts up to 100 items with positive safe-integer quantities. It loads stored prices, excludes missing/archived/out-of-stock products, caps quantity to stock and returns warnings. Coupons support percentage discounts, minimum spend, optional caps, free shipping and usage limits. The schema has no coupon-expiry or fixed-amount-discount field.

RULE	IMPLEMENTATION
Stored money	Integer minor units; serializers also expose major units.
Free/reduced shipping	Subtotal >=7500 minor units ($75), or free-shipping coupon.
Standard shipping	499 minor units below threshold; otherwise 0.
Express shipping	1299 minor units below threshold; otherwise 499.
Tax	Round 8% of max(0, subtotal - discount); application rule, not configurable tax engine.
Total	max(0, subtotal - discount) + delivery + tax; 0 for empty valid quote.
Currency	Order/Stripe creation uses USD. Display controls are not proof of multi-currency settlement.

Inventory and order lifecycle

POST /api/orders recomputes a quote, optionally compares expectedTotalMinor, and associates an idempotency key with a payload hash. A transaction conditionally decrements stock, records coupon use, writes order/item/address snapshots and status history, clears ordered cart products and saves the checkout record. Retry the same logical order with the same key; omitted keys generate new ones.

Stages: Order Placed, Confirmed, Packed, Shipped, Out for Delivery, Delivered. Backward transitions are rejected; forward jumps are allowed. COD begins Pending, and an admin can collect payment. Stripe uses a separate pending-order/session flow and signed webhooks. Payment-integrity gaps are in section 21.

Evidence: server/modules/catalog; server/modules/checkout/pricingService.ts; server/modules/orders; server/modules/payments

08 Database design and integrity

The Prisma schema defines 17 PostgreSQL models. IDs are text, generally uuid() defaults; Product.id is supplied by application/seed code. Media, variants and specifications use JSON. Monetary values use integer minor units. Role and SellerStatus are enums; payment and fulfillment states are strings enforced by application logic.

AREA	MODELS
Identity and sellers	User, Session, Address, SellerApplication
Catalog and shopping	Product, CartItem, SavedItem, WishlistItem, Review
Checkout and orders	Order, OrderItem, OrderStatusHistory, CheckoutRequest
Marketing	Coupon, CouponRedemption, NewsletterSubscription
Infrastructure	HealthCheck

Keys, ownership and deletion

Unique keys protect user email, session token hash, coupon code, order number and checkout idempotency key. Cart/saved variants use userId + productId + selectedColor + selectedSize uniqueness. Wishlist is unique per user/product; Review per product/user. CheckoutRequest.orderId is optional and unique, giving an order at most one ledger record.

User deletion cascades to sessions, addresses and shopping/review/checkout data, but orders use Restrict. Product deletion cascades to shopping/review rows and sets historical OrderItem.productId to null. SellerApplication.userId also uses SetNull. Order items/status history cascade with their order; coupon deletion is restricted by redemptions.

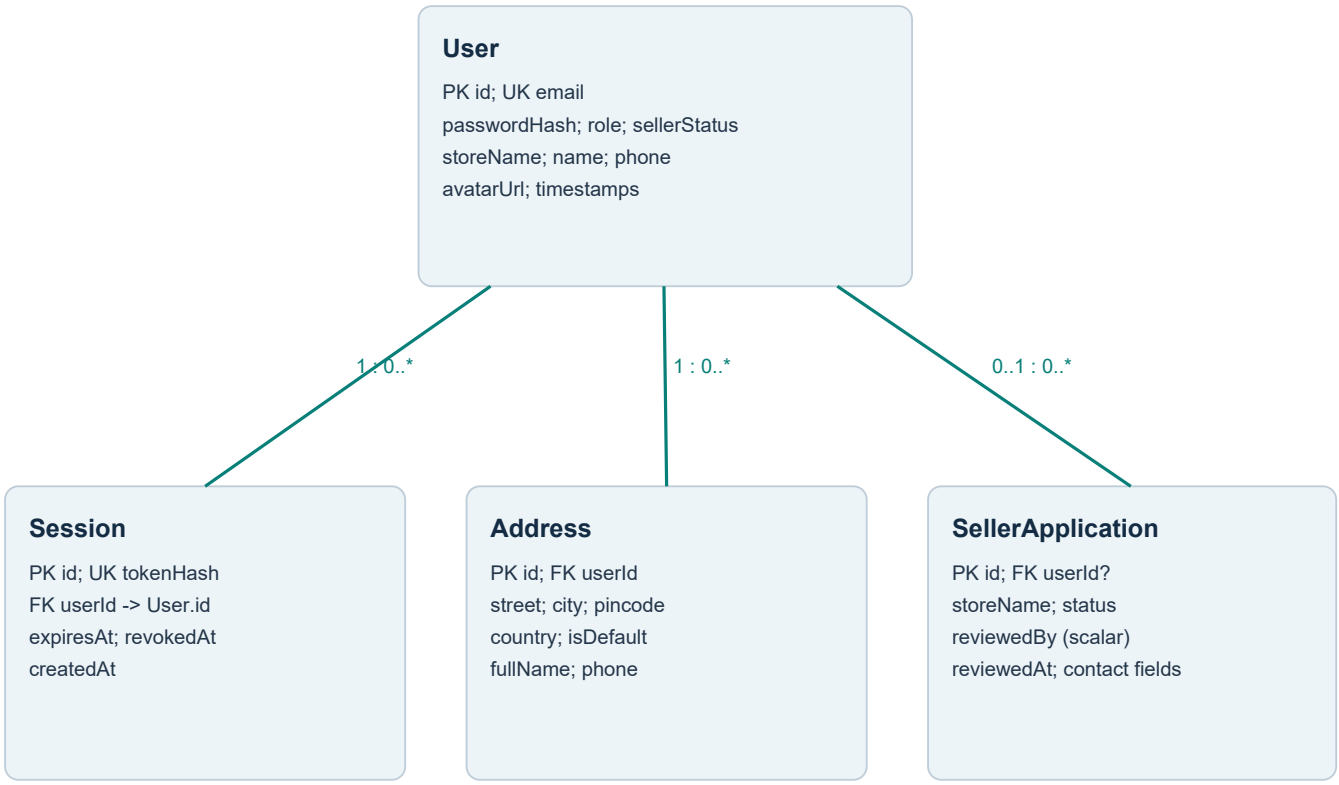
Indexes and snapshots

Indexes cover role, session expiry, owner IDs, category/archive flags, order user/date/status and review product/date. Shipping addresses and item names/prices are snapshots. CouponRedemption.orderId, SellerApplication.reviewedBy and OrderStatusHistory.actor are scalar fields, not declared foreign keys.

Migration gap: The only checked-in migration defines Role with CUSTOMER and ADMIN and lacks newer seller application/status additions. The ER diagrams describe schema.prisma, not a verified live database. A clean installation using only prisma migrate deploy will not necessarily match the schema; a reviewed follow-up migration is required.

Evidence: prisma/schema.prisma; prisma/migrations/20261019070107_init_commerce_schema/migration.sql

09 ER diagram: identity and governance

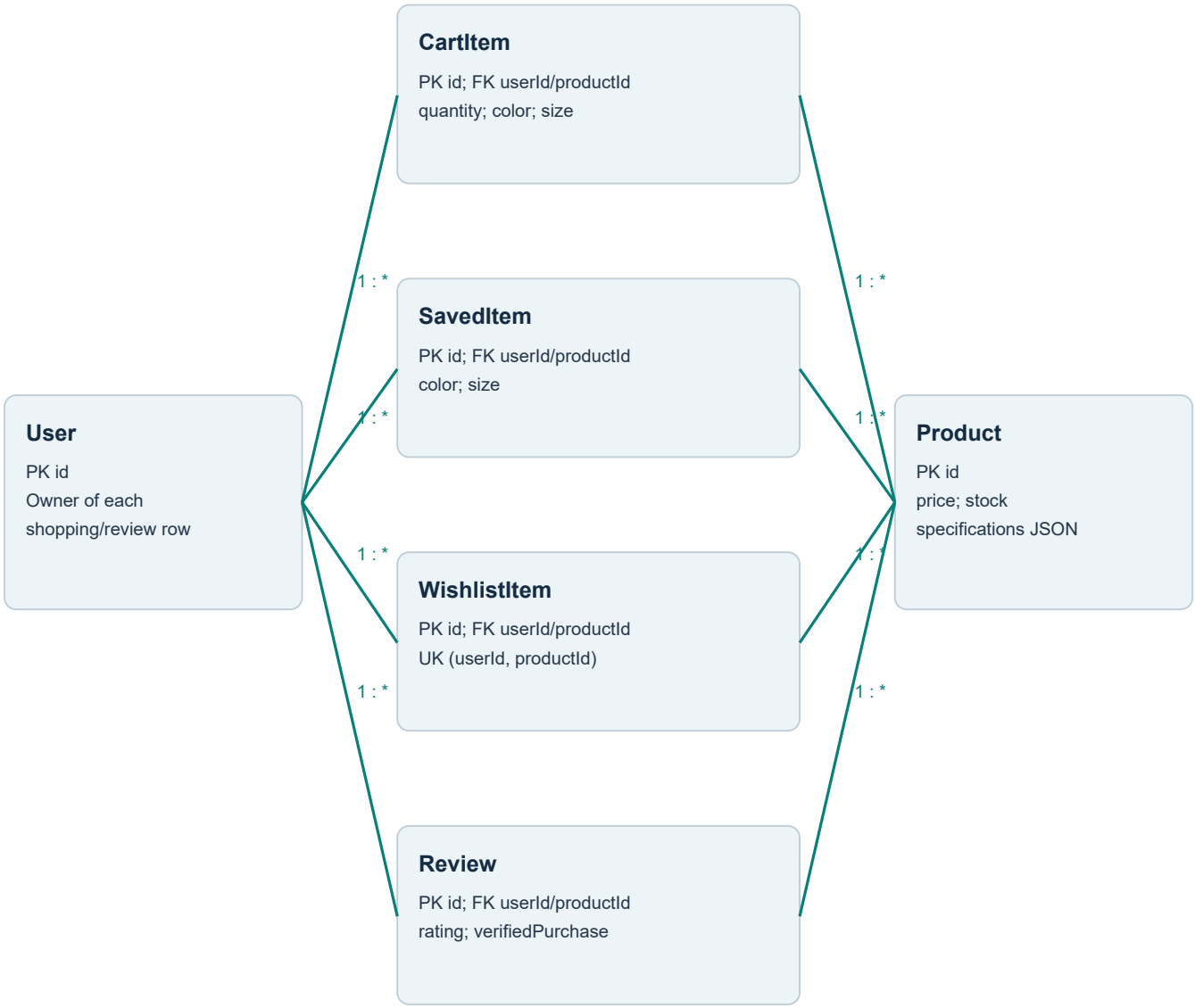


Cardinalities read from User to child. Each Session and Address belongs to exactly one User. Each SellerApplication has zero or one User; a User can have many applications. A deleted user leaves application history with a null link.

STANDALONE MODEL	ROLE
NewsletterSubscription	PK id; UK email; isActive; subscribedAt. No User foreign key.
HealthCheck	PK id; createdAt; optional note. The readiness endpoint itself performs a connectivity query.

Evidence: prisma/schema.prisma; server/modules/health/healthRoutes.ts

10 ER diagram: catalog and customer state

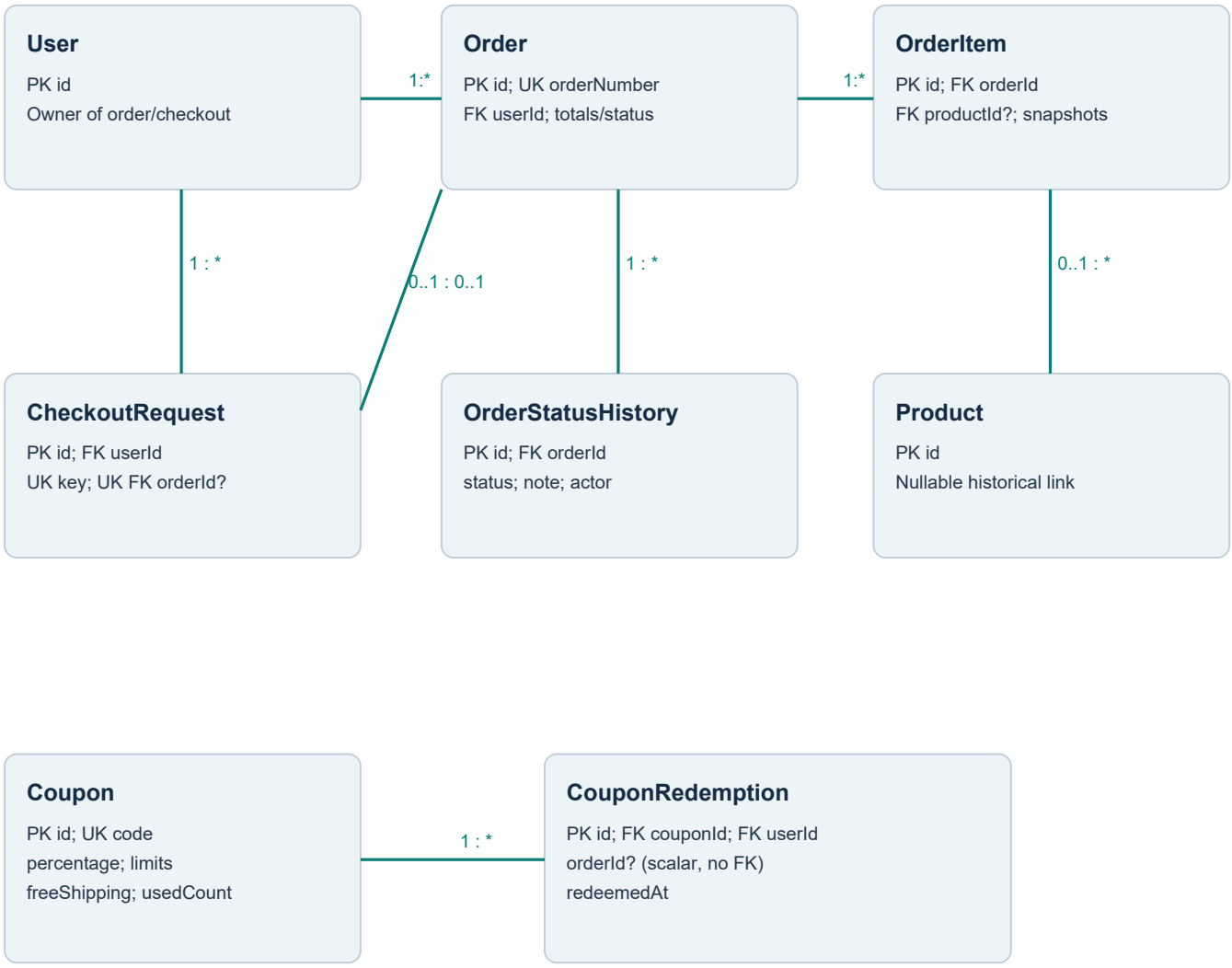


Every middle entity references one User and one Product. A User/Product can have zero or many such rows. CartItem and SavedItem uniqueness also includes color/size. Review is upserted per user/product pair.

Seller ownership is embedded in Product.specifications JSON (sellerId/sellerEmail/storeName). There is no relational Product.sellerId foreign key. This affects query indexing, integrity, deletion policy and consistent seller authorization.

Evidence: prisma/schema.prisma; server/modules/admin; server/modules/catalog

11 ER diagram: orders and promotions



User also has a 1-to-many relation to CouponRedemption, omitted from the drawing for readability. Product references can become null while order item snapshots remain. An Order has at most one CheckoutRequest; a pending request can have no order. Stripe session creation does not create that ledger record.

Order.couponCode and CouponRedemption.orderId are scalar values, not a relational Order-to-Coupon link. Order.shippingAddress is a JSON snapshot, not an Address foreign key. Solid lines represent actual Prisma relations.

Evidence: prisma/schema.prisma; orderRoutes.ts; paymentRoutes.ts

12 Data dictionary (1/5)

All persisted scalar fields. ? = nullable; PK = primary key; UK = unique key; FK = foreign key. Relations are shown in the ER diagrams.

User

id : String @id @default(uuid())
email : String @unique
passwordHash : String
name : String
phone : String?
avatarUrl : String?
role : Role @default(CUSTOMER)
sellerStatus : SellerStatus?
storeName : String?
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

Constraints/indexes: @@index([email]); @@index([role])

Session

id : String @id @default(uuid())
tokenHash : String @unique
userId : String
expiresAt : DateTime
revokedAt : DateTime?
createdAt : DateTime @default(now())

Constraints/indexes: @@index([userId]); @@index([expiresAt])

Address

id : String @id @default(uuid())
userId : String
fullName : String
phone : String
street : String
city : String
state : String
pincode : String
country : String @default("India")
type : String @default("Home")
isDefault : Boolean @default(false)
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

Constraints/indexes: @@index([userId])

Evidence: prisma/schema.prisma

12 Data dictionary (2/5)

All persisted scalar fields. ? = nullable; PK = primary key; UK = unique key; FK = foreign key. Relations are shown in the ER diagrams.

SellerApplication

id : String @id @default(uuid())
userId : String?
storeName : String
legalName : String
email : String
phone : String
businessAddress : String
category : String
taxId : String?
description : String?
status : SellerStatus @default(PENDING)
rejectionReason : String?
reviewedAt : DateTime?
reviewedBy : String?
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

Constraints/indexes: @@index([email]); @@index([status])

Product

id : String @id
name : String
brand : String
category : String
description : String
price : Int
originalPrice : Int
discount : Int @default(0)
rating : Float @default(0)
reviewCount : Int @default(0)
stock : Int @default(0)
images : Json
colors : Json?
sizes : Json?
specifications : Json
isTrending : Boolean @default(false)
isBestSeller : Boolean @default(false)
isNewArrival : Boolean @default(false)
isDealOfDay : Boolean @default(false)
isArchived : Boolean @default(false)
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

Constraints/indexes: @@index([category, isArchived]); @@index([isArchived, isTrending]); @@index([isArchived, isBestSeller])

Evidence: prisma/schema.prisma

12 Data dictionary (3/5)

All persisted scalar fields. ? = nullable; PK = primary key; UK = unique key; FK = foreign key. Relations are shown in the ER diagrams.

CartItem

id : String @id @default(uuid())
userId : String
productId : String
quantity : Int @default(1)
selectedColor : String @default("")
selectedSize : String @default("")
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

Constraints/indexes: @@unique([userId, productId, selectedColor, selectedSize]); @@index([userId])

SavedItem

id : String @id @default(uuid())
userId : String
productId : String
selectedColor : String @default("")
selectedSize : String @default("")
createdAt : DateTime @default(now())

Constraints/indexes: @@unique([userId, productId, selectedColor, selectedSize]); @@index([userId])

WishlistItem

id : String @id @default(uuid())
userId : String
productId : String
createdAt : DateTime @default(now())

Constraints/indexes: @@unique([userId, productId]); @@index([userId])

Review

id : String @id @default(uuid())
productId : String
userId : String
rating : Int
title : String
comment : String
verifiedPurchase : Boolean @default(false)
helpfulCount : Int @default(0)
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

Constraints/indexes: @@unique([productId, userId]); @@index([productId, createdAt])

Evidence: prisma/schema.prisma

12 Data dictionary (4/5)

All persisted scalar fields. ? = nullable; PK = primary key; UK = unique key; FK = foreign key. Relations are shown in the ER diagrams.

Order

id : String @id @default(uuid())
orderNumber : String @unique
userId : String
customerName : String
customerEmail : String
customerPhone : String
currency : String @default("USD")
subtotal : Int
discount : Int @default(0)
deliveryFee : Int @default(0)
tax : Int @default(0)
total : Int
couponCode : String?
shippingAddress : Json
deliveryMethod : String @default("Standard")
paymentMethod : String
paymentStatus : String @default("Pending")
transactionId : String?
status : String @default("Order Placed")
trackingNumber : String?
estimatedDeliveryDate : DateTime?
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

Constraints/indexes: @@index([userId, createdAt]); @@index([orderNumber]); @@index([status])

OrderItem

id : String @id @default(uuid())
orderId : String
productId : String?
name : String
image : String
selectedColor : String?
selectedSize : String?
unitPrice : Int
quantity : Int
totalPrice : Int

Constraints/indexes: @@index([orderId])

Evidence: prisma/schema.prisma

12 Data dictionary (5/5)

All persisted scalar fields. ? = nullable; PK = primary key; UK = unique key; FK = foreign key. Relations are shown in the ER diagrams.

OrderStatusHistory

id : String @id @default(uuid())
orderId : String
status : String
note : String?
actor : String?
createdAt : DateTime @default(now())

Constraints/indexes: @@index([orderId])

CheckoutRequest

id : String @id @default(uuid())
idempotencyKey : String @unique
userId : String
payloadHash : String
orderId : String? @unique
status : String @default("PENDING")
createdAt : DateTime @default(now())

Constraints/indexes: @@index([idempotencyKey])

Coupon

id : String @id @default(uuid())
code : String @unique
description : String
discountPercentage : Int @default(0)
minSpend : Int @default(0)
maxDiscount : Int?
freeShipping : Boolean @default(false)
usageLimit : Int?
usedCount : Int @default(0)
isActive : Boolean @default(true)
createdAt : DateTime @default(now())
updatedAt : DateTime @updatedAt

CouponRedemption

id : String @id @default(uuid())
couponId : String
userId : String
orderId : String?
redeemedAt : DateTime @default(now())

Constraints/indexes: @@index([couponId]); @@index([userId])

NewsletterSubscription

id : String @id @default(uuid())
email : String @unique
isActive : Boolean @default(true)
subscribedAt : DateTime @default(now())

HealthCheck

id : String @id @default(uuid())
createdAt : DateTime @default(now())
note : String?

Evidence: prisma/schema.prisma

13 API reference (1/5)

Identity, profile and health

METHOD / PATH	ACCESS	PURPOSE
POST **/api/auth/register**	Public	Create customer and session: name, email, password, phone.
GET **/api/auth/csrf-token**	Public	Issue/read CSRF cookie and return token.
POST **/api/auth/login**	Public	Verify email/password and issue session; IP+email limiter.
POST **/api/auth/logout**	Public	Revoke current session if present and clear cookie; no requireAuth guard.
GET **/api/auth/me**	Public	Optional-session probe; current user or unauthenticated state.
GET **/api/health**	Public	Process status, uptime and timestamp.
GET **/api/health/db**	Public	Database SELECT 1 readiness and latency.
GET **/api/me**	Signed-in owner	Get current user profile and addresses
PATCH **/api/me**	Signed-in owner	Update allowlisted profile fields (name, phone, avatarUrl)
GET **/api/me/addresses**	Signed-in owner	Get all addresses for current user
POST **/api/me/addresses**	Signed-in owner	Add new address
PATCH **/api/me/addresses/:id**	Signed-in owner	Update existing address
PATCH **/api/me/addresses/:id/default**	Signed-in owner	Set address as default
DELETE **/api/me/addresses/:id**	Signed-in owner	Delete address and preserve single default if needed

Profile/address routes scope by the session user. Profile PATCH allowlists name, phone and avatarUrl. Address writes support fullName, phone, street, city, state, pincode, country, type and isDefault.

Evidence: server/modules/auth/authRoutes.ts; server/modules/health/healthRoutes.ts; server/modules/users/userRoutes.ts

13 API reference (2/5)

Catalog, marketing and quotation

METHOD / PATH	ACCESS	PURPOSE
GET **/api/products**	Public	Search, filter, sort, paginate
GET **/api/products/:id**	Public	Single product details
GET **/api/products/:id/reviews**	Public	Public product reviews
POST **/api/products/:id/reviews**	Signed in	Authenticated customer review with verified purchase check
POST **/api/products/:id/reviews/:reviewId/helpful**	Public	Increment helpful count for a review
POST **/api/quote**	Public	Authoritative pricing quote calculation
POST **/api/checkout/quote**	Public	Authoritative pricing quote calculation
GET **/api/coupons**	Public	List all active public coupons
GET **/api/checkout/coupons**	Public	List all active public coupons
POST **/api/coupons/validate**	Public	Validate a specific coupon against subtotal or current cart
POST **/api/checkout/coupons/validate**	Public	Validate a specific coupon against subtotal or current cart
POST **/api/newsletter/subscribe**	Public	Subscribe email to newsletter
GET **/api/newsletter/status**	Public	Optional check for active subscription

Checkout router has two mounts: /api/quote aliases /api/checkout/quote; coupons and coupons/validate exist under both /api and /api/checkout. Public writes still pass global request policy.

Evidence: server/modules/catalog/catalogRoutes.ts; server/modules/checkout/checkoutRoutes.ts; server/modules/newsletter/newsletterRoutes.ts

13 API reference (3/5)

Cart, saved items and wishlist

METHOD / PATH	ACCESS	PURPOSE
GET **/api/cart**	Signed-in owner	Get user's active cart with verified live prices & stock
POST **/api/cart/items**	Signed-in owner	Add item to cart
PATCH **/api/cart/items/:id**	Signed-in owner	Update item quantity
DELETE **/api/cart/items/:id**	Signed-in owner	Delete single item
DELETE **/api/cart**	Signed-in owner	Clear entire cart
POST **/api/cart/merge**	Signed-in owner	Safe guest cart merge upon login
GET **/api/saved-items**	Signed-in owner	List saved items for user
POST **/api/saved-items**	Signed-in owner	Save item for later (optionally moving atomically from cart)
POST **/api/saved-items/:id/move-to-cart**	Signed-in owner	Move saved item to cart atomically
DELETE **/api/saved-items/:id**	Signed-in owner	Remove saved item
POST **/api/saved-items/merge**	Signed-in owner	Merge guest saved items
GET **/api/wishlist**	Signed-in owner	Get current user's wishlist items
POST **/api/wishlist/toggle**	Signed-in owner	Toggle product in wishlist
POST **/api/wishlist/items**	Signed-in owner	Add to wishlist
DELETE **/api/wishlist/items/:productId**	Signed-in owner	Remove from wishlist
DELETE **/api/wishlist/:productId**	Signed-in owner	Alias for remove
DELETE **/api/wishlist**	Signed-in owner	Clear wishlist
POST **/api/wishlist/merge**	Signed-in owner	Merge guest wishlist into user's account

Add-to-cart uses POST /api/cart/items, not POST /api/cart. Cart/saved IDs identify owned rows; wishlist deletion identifies productId. Merge routes accept items arrays.

Evidence: server/modules/cart/cartRoutes.ts; server/modules/saved/savedRoutes.ts; server/modules/wishlist/wishlistRoutes.ts

13 API reference (4/5)

Orders, payments and merchant operations

METHOD / PATH	ACCESS	PURPOSE
GET **/api/admin/products**	Admin / super admin	List products created by the current merchant
POST **/api/admin/products**	Admin / super admin	Add product linked to current merchant
POST **/api/admin/upload**	Admin / super admin	Upload product image to Cloudinary
PATCH **/api/admin/products/:id**	Admin / super admin	Update product
DELETE **/api/admin/products/:id**	Admin / super admin	Soft-archive product safely
GET **/api/admin/orders**	Admin / super admin	List orders isolated to current merchant
GET **/api/admin/metrics**	Admin / super admin	Authoritative aggregated dashboard metrics (isolated to current merchant)
GET **/api/admin/customers**	Admin / super admin	List registered customer accounts with order metrics (isolated to current merchant)
POST **/api/orders**	Signed in	Create order with transactional inventory decrement & idempotency
GET **/api/orders**	Signed in	List current user's orders
GET **/api/orders/:id**	Signed in	Get order by ID or orderNumber (owner or admin only)
PATCH **/api/orders/:id/status**	Admin / super admin	Admin order status transition & tracking update
PATCH **/api/orders/:id/collect-payment**	Admin / super admin	Admin payment collection for COD
GET **/api/payments/config**	Public	Expose public gateway configuration
POST **/api/payments/create-checkout-session**	Signed in	Create authenticated hosted Stripe Checkout session
POST **/api/payments/webhook**	Public	Stripe Webhook handler with verified signatures

Stripe webhook is public transport with signature verification. Admin access labels describe the current role guard, not merchant ownership enforcement. Upload: multipart image field, one file, <=10 MB.

Evidence: server/modules/admin/adminRoutes.ts; server/modules/orders/orderRoutes.ts; server/modules/payments/paymentRoutes.ts

13 API reference (5/5)

Seller governance

METHOD / PATH	ACCESS	PURPOSE
POST **/api/super-admin/seller-apply**	Public / optional session	Apply to become a marketplace seller / admin
GET **/api/super-admin/my-application**	Signed-in owner	Check status of current seller application
GET **/api/super-admin/metrics**	Super admin	Overall administrative summary
GET **/api/super-admin/applications**	Super admin	List all seller applications
PATCH **/api/super-admin/applications/:id/approve**	Super admin	Approve seller application & promote to ADMIN
PATCH **/api/super-admin/applications/:id/reject**	Super admin	Reject seller application
PATCH **/api/super-admin/applications/:id/suspend**	Super admin	Suspend verified seller store
PATCH **/api/super-admin/applications/:id/unsuspend**	Super admin	Reinstate suspended seller store
GET **/api/super-admin/admins**	Super admin	Directory of all platform administrators & sellers
PATCH **/api/super-admin/admins/:id/status**	Super admin	Toggle suspension or role of admin

my-application uses the authenticated owner. Status/role changes require SUPER_ADMIN; admin status accepts status/sellerStatus and role. Approval/rejection records review metadata.

Evidence: server/modules/superadmin/superAdminRoutes.ts

14 API contracts and examples

Protocol and envelopes

Ordinary writes use JSON and X-Requested-With: XMLHttpRequest. Browser Origin must match the allowed origin; non-browser clients without Origin still need the header. The frontend includes credentials and a CSRF cookie token when available. Session cookie: inbox_session_id. Success usually returns { data: ... }; errors return { error: { code, message, requestId? } }. Upload and webhook bodies have special formats.

OPERATION	KEY INPUT / OUTPUT
Catalog query	category, search, minPrice, maxPrice, sort, page, pageSize and merchandising flags -> products, total, page, pageSize, totalPages.
Cart	productId, quantity, selectedColor?, selectedSize?; update quantity by item ID. Merge takes items array.
Quote	items? [{productId, quantity, selectedColor?, selectedSize?}], deliveryMethod?, couponCode? -> totals with Minor equivalents, warnings, couponError?.
Order create	items?, shippingAddress, deliveryMethod?, paymentMethod?, couponCode?, expectedTotalMinor?, customer fields; idempotency-key header -> order, replayed? on retry.
Stripe session	items, shippingAddressId or shippingAddress, deliveryMethod?, couponCode? -> sessionId, url, orderNumber.
Order operations	status, note?, trackingNumber? for status PATCH; note? for collection. ID accepts internal ID or orderNumber.
Product write	name, brand, category, description, price, originalPrice, discount, stock, images, colors, sizes, specifications and feature flags. Price input is major units.
Review	rating, title?, comment -> upserted review and recomputed average.
Seller application	name?, storeName, legalName?, email, phone, businessAddress, category?, taxId?, description?, password when linking/creating account.

```
GET /api/products?category=Electronics&sort=price_asc&pageSize=20

POST /api/checkout/quote
X-Requested-With: XMLHttpRequest
Content-Type: application/json

{"items":[{"productId":"REAL_PRODUCT_ID","quantity":1}],
 "deliveryMethod":"Standard"}
```

Common statuses: 200 success; 201 created/upserted; 400 invalid input/business rule; 401 missing/expired session; 403 forbidden/CSRF; 404 missing resource; 409 conflict; 413 oversized body; 429 login limit; 503 unavailable database/service.

15 Fresh verification results

Executed 28 October 2026 against revision 91ac469 on Windows. Output retained under tmp/pdfs/. This is local verification, not deployment or payment certification.

COMMAND / TOOL	RESULT	MEANING
npm run lint | TypeScript	PASS | exit 0	Project/frontend type check, not ESLint.
npm run server:typecheck	PASS | exit 0	Backend type check.
npm run test:codeant | Node	7 passed; 0 failed	Quantity and origin/header policy regressions.
npm test | Vitest 5.0.5	93 passed; 5 failed / 98	8 files passed; 1 failed with five loopback EACCES errors.
npm run build | Vite 8.3.4	PASS | 16.06 s	Main JS 1,049.30 kB; 242.40 kB gzip; chunk warning.
npm run server:build | esbuild	PASS | 33 ms	Node bundle about 114.0 kB.

Meaning of the five failures

src/tests/security/codeantApi.test.ts failed when Supertest connected to transient 127.0.0.1 ports. EACCES occurred before HTTP assertions completed. Checks cover login origin/header protection, private seller status, existing-account password isolation, negative merge quantities and async validation errors. These are environment-blocked verifications, not application assertion failures and not passes.

Coverage and scope

Coverage was not measured. V8 configuration includes src/services/** and src/context/**, with thresholds of 70% lines, 70% functions and 60% branches. These are targets, not achieved percentages. Service tests often mock apiClient; checkoutFlow is a mocked service scenario. Some validation tests define helper logic locally. Passing them does not prove full live API/database/payment coverage.

No fresh Playwright, k6, database acceptance, Docker boot, Stripe transaction, Cloudinary upload or restore drill was executed. Existing evidence is separate. Application source was not changed for this report.

Evidence: tmp/pdfs/*.log; vitest.config.ts; src/tests; tests/codeant-security.test.mjs

16 Saved browser and performance evidence

Playwright | saved machine-readable result

The embedded report.json in playwright-report/index.html records 4 passing/expected tests, 0 unexpected, 0 flaky and 0 skipped; Chromium and 4 workers. Start: 28 October 2026, 10:34:48 IST; duration 7.52 seconds. test-results/.last-run.json also records passed. This run was not repeated.

SPEC	RESULT	SCENARIO
auth.spec.ts	1 passed	Account creation/authentication flow
cart.spec.ts	1 passed	Add item and inspect cart count
storefront.spec.ts	2 passed	Homepage/catalog and search

k6 | historical report figures

METRIC	NORMAL	SPIKE
Duration / peak VUs	40 s / 50	55 s / 500
Requests / throughput	1,065 / 26.15 req/s	8,533 / 154.25 req/s
Average latency	4.51 ms	986.74 ms
p95 / p99	7.87 / 9.33 ms	2.14 / 2.18 s
HTTP failure metric	0.00%	4.05%
Checks reported passed	100%	100%

Values are from PROJECT_EXECUTIVE_SUMMARY.md and load-tests/README.md. No independent raw k6 export, host configuration, database size or time-series evidence was found in the inspected result files. These are historical claims, not new measurements or production capacity guarantees.

100% checks can coexist with HTTP failures: the spike login check accepts 200, 400, 401 and 429. It does not mean every request completed a business operation. The spike uses sort=price-asc, but the API expects price_asc, so that branch falls back to featured sorting.

Evidence: playwright-report/index.html; test-results/.last-run.json; PROJECT_EXECUTIVE_SUMMARY.md; load-tests/*.js

16 Test tools and profiles

TOOL / ASSET	COVERAGE	STATUS
Vitest + jsdom	Cart calculations, services, ProductCard, auth validation, regressions, mocked checkout and uploads; API security uses Supertest.	Fresh 93/98; five EACCES.
Node runner	Imported cart sanitization and requestPolicy functions.	Fresh 7/7 passed.
Playwright	Four browser scenarios; HTML, retry traces and failure screenshot/video configured.	Saved 4/4 passed.
Backend task scripts	12 scripts: foundation, auth, profile, catalog, cart, pricing, orders, operations, reviews, payments, security and audit.	Present; not run here.
k6 normal	20 -> 50 -> 0 VUs / 40 s; p95<1000 ms, p99<2500 ms, errors<1%.	Historical figures.
k6 spike	20 -> 500 -> 20 -> 0 / 55 s; p95<3000 ms, p99<4000 ms, errors<5%.	Historical figures.
k6 stress	20 -> 60 -> 120 -> 200 -> 0 / 90 s; p95<2500 ms, p99<3500 ms, errors<5%.	No verified result.
k6 soak	30 VUs for 5 min plus 30 s ramps; p95<1500 ms, p99<3000 ms, errors<1%.	No verified result.
k6 endpoints	20 -> 40 -> 0 / 40 s; p95<1000 ms, p99<2000 ms, errors<2%.	No verified result.

Playwright starts/reuses Vite only; live flows require the backend/database. BASE_URL changes the browser target, but webServer still checks localhost:3000. k6 scripts accept TARGET_URL; the wrapper preflight checks localhost:4000. Use disposable data for account/order-writing tests.

Evidence: playwright.config.ts; vitest.config.ts; server/tests; load-tests/run-load-tests.ts; load-tests/*.js

17 CodeAnt analysis and quality

The supplied JSON identifies MayankGodhani28/inbox-ecommerce-store, branch Mayank, analyzed 23 October 2026 at 05:14 PM (timezone not recorded), with 11,366 lines analyzed. This is a saved scan, not a current rescan.

CATEGORY	RECORDED RESULT
SAST	1 HIGH issue; 19 findings classified as false positives.
Secrets	0 detected in the saved scan.
Infrastructure	0 reported issues.
Composition analysis	0 reported vulnerabilities; 246 healthy packages scanned.
Code quality	278 antipatterns; 3 complex functions; 41 missing docstrings; 5 duplicate-code groups; 0 dead-code issues.
SBOM licenses	246 components; 246 permissive; 0 non-permissive.

High-severity quantity finding

The scan flags quantity validation in src/context/CartContext.tsx (CWE-20). Current source includes isPositiveQuantity and sanitizeCart in src/utils/cartValidation.ts, requiring positive safe integers and sanitizing persisted entries. The quote service also validates quantities. Seven fresh Node regressions passed. This shows remediation work, but no updated scan was run to close the historical finding.

Interpretation and CI

Zero reported secrets/vulnerabilities means none were reported within that saved scope at that time. It is not assurance that current dependencies, backups or deployment are risk-free. False-positive classifications and scanner risk scores were not independently validated.

GitHub Actions runs on pushes to Mayank or manual dispatch: Node 24, npm ci, Prisma generation, Node security tests, frontend/backend checks, Vitest and both builds. It does not provision PostgreSQL, execute Playwright/k6 or collect coverage. Remote workflow results were not fetched.

Evidence: MayankGodhani28-inbox-ecommerce-store-analysis-report.json; src/utils/cartValidation.ts; .github/workflows/codeant-verification.yml

18 Local setup and runbook

Use the project root, Node >=22.20.0, npm and PostgreSQL. Commands use PowerShell. Replace placeholders locally and preserve any existing .env.

1. Install and configure

```
npm ci
Copy-Item .env.example .env
# Edit .env for database and optional integrations.
npm run prisma:generate
```

DATABASE_URL is required; URL-encode special password characters. PORT defaults to 4000 and CLIENT_ORIGIN to http://localhost:3000. Clear placeholder Stripe values unless using valid test keys.

2. Align the development database

```
# Local/disposable development database only.
npm run prisma:migrate -- --name align_current_schema
npm run db:seed
npm run prisma:studio
```

Migration history is behind the schema. migrate dev may detect drift or request a reset; do not reset valuable data. Back up and review SQL. Seeding upserts existing catalog prices and stock, so it is not routine production startup.

3. Start both processes

```
# Terminal A: http://localhost:4000
npm run server:dev

# Terminal B: http://localhost:3000
npm run dev
```

4. Check readiness

```
Invoke-RestMethod http://localhost:4000/api/health
Invoke-RestMethod http://localhost:4000/api/health/db
```

Open / for shoppers, /admin for merchants, /super-admin for governance and /sell for onboarding. Liveness alone does not prove database readiness.

Evidence: package.json; .env.example; server/config/env.ts; prisma/migrations; server/db/seed.ts

18 Configuration and provisioning

VARIABLE / GROUP	ACTUAL PURPOSE
DATABASE_URL	Required Prisma PostgreSQL URL.
PORT / NODE_ENV	Port/environment; production Secure cookies require HTTPS.
CLIENT_ORIGIN	Allowed origin and Stripe redirect base.
VITE_API_URL	Vite proxy target from process.env; apiClient uses relative URLs. Export in shell to override reliably.
STRIPE_SECRET_KEY	Server-only payment credential.
STRIPE_PUBLISHABLE_KEY	Public gateway configuration.
STRIPE_WEBHOOK_SECRET	Raw-body webhook verification.
CLOUDINARY_* / CLOUDINARY_URL	Cloud name, key/secret or full URL for upload.
ADMIN_EMAIL / ADMIN_PASSWORD / ADMIN_NAME / ADMIN_PHONE	bootstrapAdmin.ts inputs; email/password mandatory.
SUPER_ADMIN_EMAIL / SUPER_ADMIN_PASSWORD	Super-admin seed; password 12+ characters, <=72 UTF-8 bytes.
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	Compose database inputs.
BASE_URL / TARGET_URL	Playwright target / k6 target.

Development account provisioning

```
$env:ADMIN_EMAIL = "admin@example.com"
$env:ADMIN_PASSWORD = "<unique-password>"
npm run admin:bootstrap

$env:SUPER_ADMIN_EMAIL = "owner@example.com"
$env:SUPER_ADMIN_PASSWORD = "<unique-12-plus-character-password>"
npx tsx server/db/seedSuperAdmin.ts
```

Bootstrap can replace a password/promote an existing account. Super-admin seed adds sample applications and adjusts a demo admin. No accounts were created here.

SESSION_SECRET is mentioned by docs/Compose but not consumed by current session code. .env.example names COMPOSE_DATABASE_URL; Compose reads DATABASE_URL. localhost inside an app container does not address its postgres service.

19 Deployment and recovery

Standalone production build

```
npm ci
npm run prisma:generate
npm run build
npm run server:build
# After reviewing complete committed migration history:
npm run prisma:deploy
$env:NODE_ENV = "production"
$env:CLIENT_ORIGIN = "https://your-store.example"
npm run server:start
```

Use HTTPS and a matching origin. Production cookies are Secure/SameSite=Strict; an HTTP-only example is not a complete authenticated deployment. vercel.json builds only the frontend, without deploying Express or an API proxy.

Container deployment and outstanding checks

```
# Review/correct configuration before deploying.
docker compose -f compose.yaml config
docker compose -f compose.yaml up -d --build
docker compose -f compose.yaml logs --tail 100 app
```

Both Compose files duplicate the stack. They declare postgres:18-alpine, a named volume, localhost database exposure, app port 4000 and health checks. Startup does not migrate/seed. The runner omits devDependencies but invokes npx prisma generate; validate CLI availability in a clean image. Verify the PostgreSQL 18 volume layout before relying on persistence. Docker was not run here.

Monitoring

Health/readiness routes and logs expose timestamp, requestId, method, URL, status and durationMs. Selected query secrets are redacted. Shutdown closes HTTP and disconnects Prisma. Central alerts/metrics storage were not verified.

Backup and restore

```
npm run db:backup
pg_dump -U postgres -d inbox_store -F c -f inbox_store.dump
# Separate clean recovery database only:
pg_restore -U postgres -d inbox_store_recovery inbox_store.dump
```

JSON backup exports nine model groups and checks structure/product counts; it does not restore data. It omits sessions, carts, saved/wishlist items, redemptions, checkout requests, seller applications and HealthCheck. Use full backups and a measured restore drill. Tracked backups may contain password hashes/personal data and were not reproduced.

20 Command reference

COMMAND	PURPOSE
npm run dev	Frontend on port 3000
npm run build	Frontend production assets
npm run preview	UI preview; no API proxy configured here
npm run lint	Project/frontend TypeScript check
npm run server:dev	Watch-mode Express
npm run server:typecheck	Backend check
npm run server:build	Node ESM bundle
npm run server:start	Run compiled server
npm run prisma:generate	Generate Prisma client
npm run prisma:migrate	Dev migration; review drift/reset prompts
npm run prisma:deploy	Apply committed migrations
npm run prisma:studio	Database browsing UI
npm run db:seed	Upsert demo catalog/coupons
npm run db:backup	Partial JSON export/check
npm run admin:bootstrap	Provision/update admin
npm run test	Vitest suite
npm run test:watch	Vitest watch
npm run test:unit	Run unit Vitest subset
npm run test:integration	Run integration Vitest subset
npm run test:component	Run component Vitest subset
npm run test:security	Run security Vitest subset
npm run test:regression	Run regression Vitest subset
npm run test:e2e	Run e2e Vitest subset
npm run test:all	Vitest verbose
npm run test:coverage	V8 coverage and thresholds
npm run test:pw	Chromium suite
npm run test:pw:ui	Playwright UI
npm run test:pw:headed	Visible browser tests
npm run test:pw:report	Saved HTML report
npm run load:normal	Run k6 normal scenario(s)
npm run load:spike	Run k6 spike scenario(s)
npm run load:stress	Run k6 stress scenario(s)
npm run load:soak	Run k6 soak scenario(s)
npm run load:endpoints	Run k6 endpoints scenario(s)
npm run load:all	Run k6 all scenario(s)
npm run test:codeant	Node security regressions

Aliases: k6:normal, k6:spike, k6:stress, k6:soak, k6:endpoints use the same runner as load:* equivalents. npm test is shorthand for npm run test.

20 Verification and smoke commands

Repeatable review sequence

```
npm run lint
npm run server:typecheck
npm run test:codeant
npm test
npm run test:coverage
npm run build
npm run server:build

# Backend/database needed for live browser workflows:
npx playwright install chromium
npm run test:pw
npm run test:pw:report
```

Performance tests

```
# Disposable local/staging data only.
npm run load:normal
npm run load:spike
npm run load:stress
npm run load:soak
npm run load:endpoints
```

To retain a machine-readable result, invoke tools/k6.exe directly with run, --summary-export=k6-normal.json and load-tests/k6-normal-load.js. On other operating systems, use a compatible k6 binary on PATH.

Read-only smoke requests

```
Invoke-RestMethod http://localhost:4000/api/products
Invoke-RestMethod http://localhost:4000/api/coupons
```

Backend acceptance scripts

Run server/tests files individually with npx tsx followed by the real path. Available task numbers: 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 14, 16. Inspect fixtures, credentials and writes before selecting the test database.

Evidence: package.json; server/tests; load-tests/run-load-tests.ts; playwright.config.ts

21 Release priorities

These findings come from source inspection, not a penetration test. Confirm them with targeted integration tests before public commerce deployment.

PRIORITY	FINDING / IMPACT	ACCEPTANCE CONDITION
Critical	POST /api/orders marks non-COD methods Paid from client input, without gateway verification.	Restrict path to COD or verify payment server-side; add negative integration tests.
High	Admin order status/collection lacks merchant ownership checks. Matching order lists expose all line items.	Scope every seller read/write and filter unrelated merchant data.
High	Session/admin guards do not enforce suspended seller status; archiving existing products does not revoke admin capability.	Block suspended writes and constrain/revoke sessions; test API directly.
High	Stripe reserves stock before remote session creation; no compensation on creation failure/expiry, and no request idempotency.	Add reservation release/expiry and idempotent session creation.
High	Failed payment lookup uses payment-intent ID while the initial transactionId stores session ID; duplicate failures can re-release stock.	Persist gateway IDs/event deduplication; verify amount, currency, paid state and once-only release.
High	Migration history lacks newer seller schema and enum fields.	Clean-database migration plus full acceptance run.
Medium	Coupon user-use uniqueness is not constrained; Stripe path does not record redemption counters.	Consistent transactional quotas across payment paths and concurrency tests.
Medium	Partial JSON backup does not actually restore data.	Full dump, restricted retention and measured restore drill.
Medium	Sample financial charts, large JS bundle and tests that partly mirror local helper logic.	Real analytics, bundle splitting and actual backend contract testing.

Evidence: server/modules/orders; server/modules/payments; server/middleware/auth.ts; prisma/migrations; server/scripts/backupAndRestore.ts; AdminDashboard.tsx

21 Security controls and boundaries

CONTROL	IMPLEMENTATION	BOUNDARY
Passwords / sessions	bcrypt (12 rounds in registration/provisioning); random 32-byte tokens; SHA-256 token hashes; 24-hour expiry.	No password reset/email verification route found.
Cookies	HttpOnly, Secure in production, SameSite Strict in production/Lax otherwise.	Needs HTTPS; does not remove all XSS/CSRF risk.
Request policy	Exact allowed origins, required X-Requested-With on unsafe requests, exact webhook exception in first guard; second CSRF layer.	Test both layers as one policy.
Login throttling	In-memory IP+email map; 5 attempts per 15 minutes.	Not shared across replicas; not general DDoS protection.
Limits	JSON/urlencoded <=1 MB; one uploaded image <=10 MB with allowed MIME types.	MIME allowlisting is not content verification.
Database	Prisma, foreign/unique keys and conditional stock updates.	No universal stock/rating CHECK constraint or formal proof of all logic safety.
Authorization	Owner filtering for personal records; admin/super-admin role guards.	Seller mutation scope/suspension need strengthening.
Observability	Request IDs, structured logs and sanitized production errors.	No complete monitoring/security-header middleware verified.

Do not treat this as PCI certification, proof of zero vulnerabilities or unlimited concurrency. Hosted payment UI reduces direct card-data handling, but payment-state integrity remains an application responsibility. Refunds, returns, seller payouts, shipping-provider integration and notification-delivery APIs are absent from the inspected route inventory.

Evidence: server/middleware; server/security/requestPolicy.ts; server/modules/auth; server/app.ts; route inventory

22 Demonstration and evaluation guide

Suggested 12-minute demonstration

TIME	SHOW	EVIDENCE
0-2 min	Purpose, stack and architecture.	Distinguish implemented scope and future work.
2-4 min	Browse/search/filter, product and cart.	Catalog request, variants, stock and guest/persisted state.
4-6 min	Login, address, quote and test COD order.	Authoritative totals, stock change and idempotency replay.
6-8 min	Merchant product management and fulfillment.	Upload configuration, status history and COD collection.
8-10 min	Super-admin seller application review.	Role/status model and transitions; explain scope gaps.
10-12 min	Tests, scan and release priorities.	Fresh logs, saved browser evidence and labeled k6 history.

Manager evaluation criteria

Assess functional breadth, role clarity, pricing/order correctness, test depth, recoverability and deployment reproducibility. Evaluate UI completeness independently from data provenance and backend authorization. Request a clean-database installation and negative tests for cross-seller access, forged paid orders, checkout retries and duplicate webhooks.

Strengths and outcome

The repository demonstrates broad commerce workflows, modular APIs, a substantial relational model, server pricing, conditional stock updates, cookie sessions, payment/media integrations, multilingual structure and multiple testing tools. This report consolidates the schema, API inventory, evidence and runbook.

A development demonstration is supported by the available evidence. Before production approval, close critical/high findings, rerun integration/browser checks on an unrestricted test host, review migrations, verify payment events and prove restoration from backup.

22 Source and result index

SOURCE	USED FOR
package.json / package-lock.json	Scripts, runtime requirement and locked versions.
src/App.tsx	Screens and browser navigation.
src/pages/AdminDashboard.tsx / SuperAdminDashboard.tsx / SellerOnboard.tsx	Panel scope, sample analytics and governance.
src/context / src/services / src/i18n	State, fetch transport and translations.
server/app.ts / server/index.ts	Mounted routes, middleware, static serving and shutdown.
server/modules/*/*Routes.ts	All HTTP operations, guards and business behavior.
server/modules/checkout/pricingService.ts	Authoritative quantity and money rules.
server/middleware / server/security	Session, role, upload and CSRF/origin controls.
prisma/schema.prisma / prisma/migrations	Models, dictionary, relationships and schema drift.
server/db / server/scripts	Seed, provisioning, backup and operations.
Dockerfile / compose.yaml / docker-compose.yml / vercel.json	Deployment configuration.
vitest.config.ts / src/tests / tests/codeant-security.test.mjs	Test scope and coverage targets.
playwright-report/index.html / test-results/.last-run.json / e2e	Saved browser results and assertions.
load-tests / PROJECT_EXECUTIVE_SUMMARY.md	Load profiles and historical figures.
README.md / DOCUMENTATION.md	Documentation comparison.
MayankGodhani28-inbox-ecommerce-store-analysis-report.json	Saved CodeAnt scan.
tmp/pdfs/frontend-typecheck.log / backend-typecheck.log / codeant-tests.log / vitest.log / frontend-build.log / backend-build.log	Fresh verification output.

Paths are relative to the project root. Findings describe the reviewed checkout, not a remote deployment or later commit. Diagrams derive from schema.prisma. No private environment values or backup records are included.